

DISEÑO Y MODELOS DE SISTEMAS SOFTWARE 2025-2026

Modelado de Servicios MCP y Agentes IA

Autores:

Javier Montero Pajuelo 77026354K

Pablo Carrasco Vázquez 09217704V

Índice

1. Descripción del dominio de la aplicación y de los objetivos del proyecto	2
2. Sintaxis Abstracta de Modelado de Servicios MCP y Agentes IA	3
2.1 Descripción de los conceptos del dominio	4
3. Sintaxis Concreta de Modelado de Servicios MCP y Agentes IA	6
3.1. Elementos Contenedores y de Arquitectura	6
3.2. Elementos de Memoria y Datos	7
3.3. Nodos de Flujo (Tareas)	7
3.4. Enlaces y Relaciones (Links)	8
3.5 Modelo de ejemplo Modelado de Servicios MCP y Agentes IA	8
4. Generación de código	9
A. Generación Web Dinámica por Tarea (tarea_[nombre_tarea].html)	10
B. Generación del Script de Base de Datos (base_datos.sql)	10
5. Extensiones Implementadas	11
6. Herramientas de desarrollo	11
7. Contenidos del archivo que se entrega	12
8. Anexo	14
Anexo 1. Planificación	14
Anexo 2. Repositorio de Github	14

1. Descripción del dominio de la aplicación y de los objetivos del proyecto

El dominio de este proyecto se sitúa en la **orquestración de sistemas multi-agente basados en Inteligencia Artificial** y la integración de servicios externos mediante el protocolo **MCP (Model Context Protocol)**.

En la actualidad, la creación de flujos de trabajo (workflows) que involucren agentes inteligentes, modelos de lenguaje (LLM) y herramientas externas es una tarea compleja que requiere conocimientos profundos de programación. Este DSL permite abstraer dicha complejidad, proporcionando un entorno visual donde se pueden diseñar procesos de negocio automatizados. El dominio abarca desde la definición de los agentes y sus roles, hasta la gestión de la "memoria" del sistema (Contextos) y la conexión con servidores que ejecutan operaciones específicas sobre datos reales.

Los objetivos principales concretos de este proyecto son:

- **Abstracción del diseño de flujos:** Permitir a usuarios inexpertos diseñar flujos de trabajo complejos mediante una interfaz gráfica, eliminando la necesidad de escribir código para la estructura del proceso.
- **Formalización mediante Metamodelo:** Definir una sintaxis abstracta sólida (Metamodelo) que garantice que los diagramas creados sean lógicamente consistentes.
- **Automatización de la persistencia:** Generar automáticamente la estructura de base de datos relacional (SQL) necesaria para soportar la memoria y los contextos de los agentes definidos en el modelo.
- **Generación de interfaces funcionales:** Crear de forma automática un prototipo de portal web (HTML/CSS) que sirva como interfaz de usuario para cada agente, permitiendo la navegación real entre las tareas diseñadas.

2. Sintaxis Abstracta de Modelado de Servicios MCP y Agentes IA

- **Regla OCL (R01_UnaTareaInicio):** Asegura que exista exactamente un punto de inicio.
- **Regla OCL (R02_UnaTareaFin):** Asegura que exista exactamente un punto de finalización.
- **Contiene:** Lista de Tarea.

Elemento (EClass): Contexto

- **Descripción:** Representa el estado o conjunto de datos persistentes compartidos durante la ejecución del workflow de un agente. Funciona como la memoria del sistema.
- **Atributos:** nombreContexto (String).
- **Contiene:** Lista de Propiedad.

Elemento (EClass): Propiedad

- **Descripción:** Representa una variable o campo de datos individual dentro de un contexto.
- **Atributos:** nombrePropiedad (String), tipoPropiedad (Enum: TipoContexto). Se incluyen atributos de seguridad avanzada (*Extensión 1*): extensionPermitida (String) y pesoMaxMB (int) para la validación proactiva en el frontend.
- **Contiene:** VersionContexto (Historial de cambios).

Elemento (EClass): VersionContexto (*Nueva clase de Extensión 1*)

- **Descripción:** Elemento clave para la trazabilidad de datos. Almacena el valor histórico de una propiedad antes de ser modificada, garantizando la auditoría de los datos en entornos de producción.
- **Atributos:** valorAnterior (String), fechaModificacion (String).
- **Referencias:** propiedadAsociada.

Elemento (EClass Abstracta): Tarea

- **Descripción:** Clase base abstracta para todos los pasos ejecutables y puntos de control dentro de un workflow.
- **Regla OCL (R10_NoBuclePropio):** self.sigue <> self. Evita que una tarea cree un bucle infinito apuntándose a sí misma.
- **Atributos:** nombreTarea (String), taskId (int), taskDesc (String).
- **Referencias:** precede, sigue, precedeElse (para gestionar el orden del flujo de ejecución).

Elemento (EClass Abstracta): TareaEjecutable (*Extiende a Tarea*)

- **Descripción:** Clase base para aquellas tareas que ejecutan una acción concreta, diferenciándose de los nodos de control puramente lógicos (Inicio/Fin).
- **Regla OCL (R09_EntradaYSalida):** Asegura que estas tareas no queden aisladas o huérfanas, obligando a que tengan al menos una conexión de entrada y una de salida.
- **Referencias:** ejecutadaPor (Referencia obligatoria a 1 Agente).

Elementos de Control: Inicio y Final (*Extienden a Tarea*)

- **Inicio:** Representa el punto de entrada del flujo. Regla OCL (R03): No puede tener enlaces entrantes (precede->isEmpty()).

- **Final:** Nodo final que indica la terminación lógica. Regla OCL (R04): No puede tener enlaces salientes (`sigue.ocllsUndefined()`).

Elemento (EClass Abstracta): **Accion, Escritura y Lectura**

- **Descripción:** Modelan las interacciones que una tarea realiza sobre un contexto (ya sea para leer datos y procesarlos, o para sobrescribir resultados).
- **Regla OCL (R12_CoherenciaContexto):** Evita la corrupción de memoria verificando que las propiedades que se van a usar pertenecen realmente al contexto al que se apunta.

Elementos de Tareas Específicas (Extienden a **TareaEjecutable**)

- **TareaUsuario:** Tarea interactiva que detiene el flujo para requerir entrada de datos manual (formularios web).
- **TareaLLM:** Invoca a un Modelo de Lenguaje utilizando un prompt específico.
- **TareaTransformacionDatos:** Reformatea un conjunto de datos en base a una regla de transformación.
- **TareaAnálisis:** Tarea de bifurcación condicional. A través de un prompt de decisión, activa el camino principal (`sigue`) o el flujo alternativo (`sigueElse`).

Comunicación Inter-Agente: **TareaEnvioContexto** y **TareaRecepcionContexto**

- **Descripción:** Permiten la exportación e importación de contextos entre distintos agentes de forma asíncrona.
- **Reglas OCL Aplicadas:** La recepción está rígidamente controlada. (R07) Debe estar precedida por un envío. (R11) Las propiedades exportadas e importadas deben ser simétricas. (R08) El agente emisor y receptor no pueden ser el mismo.

Elemento (EClass): **ServerMCP y OperacionMCP**

- **Descripción:** Representan la infraestructura externa del protocolo MCP. Un **ServerMCP** agrupa herramientas disponibles, y una **OperacionMCP** es una función invocable de forma remota.
- **Elemento TareaServerMCP:** Tarea automatizada del agente encargada de invocar remotamente la **OperacionMCP** asociada enviando un prompt de ejecución.

3. Sintaxis Concreta de Modelado de Servicios MCP y Agentes IA

La sintaxis concreta gráfica de este Lenguaje Específico de Dominio (DSL) ha sido desarrollada utilizando el framework EuGENia sobre GMF. Permite a los diseñadores de sistemas crear flujos de trabajo arrastrando y conectando elementos en un lienzo visual. A continuación se detallan las representaciones gráficas de cada concepto:

3.1. Elementos Contenedores y de Arquitectura

- **Concepto: ServiceMCP (Lienzo)**
 - Representación Gráfica: Actúa como el lienzo o diagrama principal (`@gmf.diagram`)

donde se instancian y organizan todos los Agentes, Workflows y Servidores del modelo.

- **Concepto: Agente**
 - Representación Gráfica: Nodo rectangular de color rosa pastel (RGB: 255, 200, 200). Cuenta con un compartimento interno de tipo lista (layout="list") para organizar visualmente los Contextos que le pertenecen.
- **Concepto: Workflow**
 - Representación Gráfica: Nodo con bordes redondeados (figure="rounded") de color verde pastel (RGB: 200, 255, 200). Posee un compartimento de diseño libre (layout="free") que actúa como un sub-lienzo donde se dibujan y conectan las Tareas.
- **Concepto: ServerMCP**
 - Representación Gráfica: Nodo rectangular de color azul claro (RGB: 200, 200, 255) con un compartimento en forma de lista para declarar las operaciones remotas disponibles.

3.2. Elementos de Memoria y Datos

- **Concepto: Contexto**
 - Representación Gráfica: Nodo rectangular de color amarillo claro (RGB: 255, 255, 200). Contiene un compartimento en formato de lista para visualizar las propiedades.
- **Concepto: Propiedad**
 - Representación Gráfica: Nodo básico listado dentro de su contexto padre. Su etiqueta muestra el nombre de la variable.
- **Concepto: OperacionMCP**
 - Representación Gráfica: Nodo listado dentro del ServerMCP correspondiente, mostrando el nombre de la operación externa invocable.

3.3. Nodos de Flujo (Tareas)

Todos los nodos de tareas muestran su identificador o nombre en el centro de la figura. Para facilitar la lectura visual del diagrama, se ha aplicado una codificación de colores semántica:

Nodos de Control de Flujo:

- **Inicio:** Nodo circular/elíptico de color verde brillante (RGB: 0, 255, 0) y tamaño fijo (30x30).
- **Final:** Nodo circular/elíptico de color rojo intenso (RGB: 255, 0, 0) y tamaño fijo (30x30).
- **TareaAnálisis:** Nodo rectangular de color amarillo pálido (RGB: 255, 255, 150).

Nodos de Procesamiento IA y Datos:

- **TareaLLM:** Nodo rectangular de color magenta claro (RGB: 255, 200, 255).
- **TareaTransformacionDatos:** Nodo rectangular de color cyan claro (RGB: 200, 255, 255).

Nodos de Interacción:

- **TareaUsuario:** Nodo rectangular de color blanco (RGB: 255, 255, 255).
- **TareaServerMCP:** Nodo rectangular de color gris claro (RGB: 200, 200, 200).

Nodos de Comunicación Inter-Agente:

- **TareaEnvioContexto:** Nodo rectangular de color verde pálido (RGB: 150, 255, 150).
- **TareaRecepcionContexto:** Nodo rectangular de color azul oscuro apastelado (RGB: 150, 150, 255).

Acciones de Tarea:

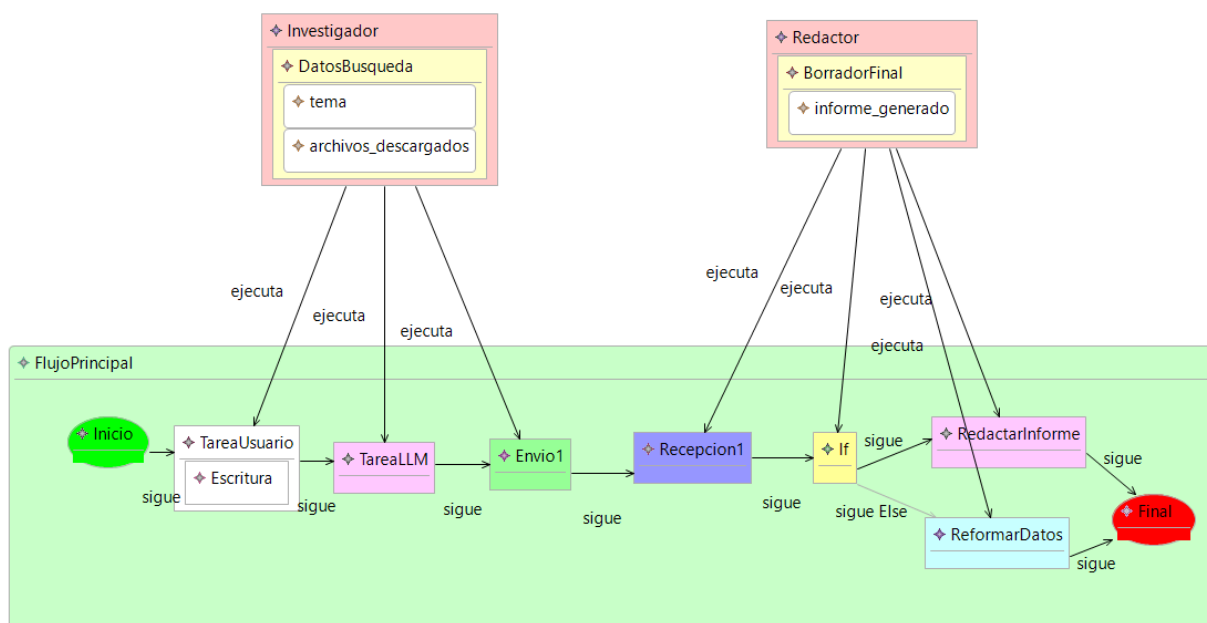
- **Escritura y Lectura:** Nodos rectangulares insertados dentro del compartimento de tipo lista de la tarea correspondiente, mostrando el nombre de la acción.

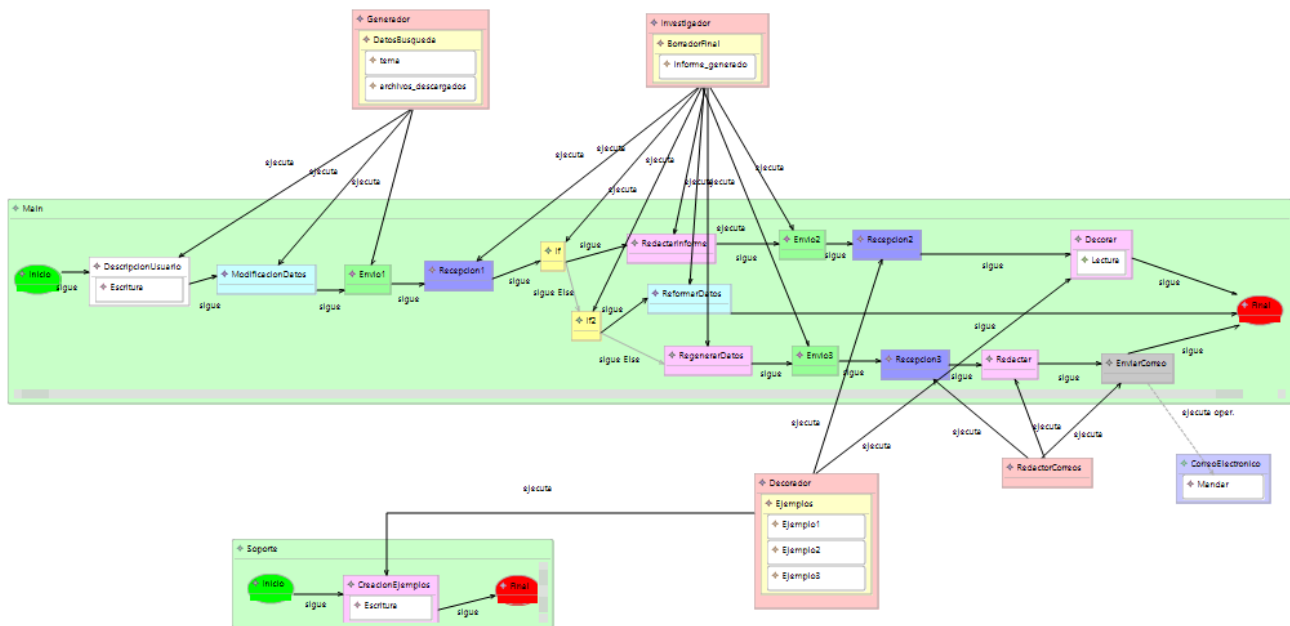
3.4. Enlaces y Relaciones (Links)

Las transiciones y asignaciones lógicas del sistema se representan mediante flechas dirigidas (@gmflink), decoradas según su propósito:

- **Referencia de Asignación (ejecuta):** Conecta un Agente con las TareaEjecutable que le pertenecen. Se dibuja como una línea continua de color negro terminada en una flecha direccional con la etiqueta textual "ejecuta".
- **Referencia de Transición Principal (sigue):** Conecta una Tarea con su sucesora natural en el flujo. Se representa como una línea continua de color negro terminada en flecha, con la etiqueta "sigue".
- **Referencia de Bifurcación Alternativa (sigue Else):** Exclusiva de la TareaAnálisis. Representa el camino negativo o secundario de la decisión. Finaliza en flecha y lleva la etiqueta "sigue Else".
- **Referencia de Invocación Remota (ejecuta oper.):** Conecta una TareaServerMCP con la OperacionMCP externa que debe disparar. Se dibuja mediante una línea discontinua (style="dash") terminada en flecha, para diferenciar visualmente que es una llamada externa y no un paso del flujo local.

3.5 Modelo de ejemplo Modelado de Servicios MCP y Agentes IA





4. Generación de código

En esta sección se describe la estructura de artefactos generada automáticamente mediante la transformación de modelos a texto con Acceleo. El proceso toma como entrada un modelo conforme a la sintaxis abstracta (McpMM) y genera un portal web funcional y un script de persistencia SQL.

La generación organiza los archivos de forma plana en el directorio raíz de destino (por ejemplo, /generado), interconectándolos de forma dinámica sin necesidad de subcarpetas. A continuación, se detalla la jerarquía y tipología de los archivos generados:

- **index.html:** Punto de entrada global del servicio. Renderiza un panel central que lista todos los agentes configurados y proporciona enlaces para acceder al perfil de cada uno, mostrando además el recuento de tareas asignadas.
- **agente_[nombre_agente].html:** Panel de control individualizado. Genera una lista con las TareaEjecutable asignadas al agente en cuestión e incluye botones de navegación directa hacia cada tarea.
- **tarea_[nombre_tarea].html:** Vistas HTML específicas generadas iterativamente para cada tarea del workflow. Su contenido varía dinámicamente según la clase (LLM, Usuario, Análisis, etc.).
- **base_datos.sql:** Script DDL que traduce los conceptos de Contexto y Propiedad a tablas relacionales de base de datos.

A. Generación Web Dinámica por Tarea (tarea_[nombre_tarea].html)

La plantilla generateTaskView(t : Tarea) evalúa el polimorfismo de las tareas mediante sentencias oclIsTypeOf para renderizar el HTML adecuado:

- **TareaUsuario:** Genera formularios <form> dinámicos iterando sobre las acciones de Escritura de la tarea. Si la propiedad es un archivo, inyecta atributos HTML5 avanzados y JavaScript (accept y onchange) extraídos del modelo para limitar la extensión y el peso máximo. Además, si la propiedad tiene histórico, renderiza una lista mostrando los cambios previos (VersionContexto).
- **TareaAnálisis (If/Else):** Genera una bifurcación en la interfaz de usuario insertando dos botones: uno enlazado a la referencia sigue (Camino Sí) y otro enlazado a la referencia sigueElse (Camino NO).
- **TareaLLM / TareaServerMCP / Transformación:** Muestran el prompt inyectado desde el modelo y simulan la ejecución de la inteligencia artificial o del servicio remoto, proporcionando un botón para continuar el flujo.

B. Generación del Script de Base de Datos (base_datos.sql)

El bloque Acceleo generateSQL(aService : ServiceMCP) recorre todos los agentes y sus contextos para crear el script de almacenamiento estructurado:

1. **Tablas Principales:** Por cada Contexto, crea una tabla con una clave primaria autoincremental (id_registro). Los atributos se generan mapeando el enumerado TipoContexto a tipos SQL nativos mediante la función auxiliar mapTipoSQL (ej: ENTERO -> INT, ARCHIVO -> VARCHAR(500)).
2. **Tablas de Historial:** De forma automática, genera una segunda tabla relacional denominada [nombre_contexto]_Historial, enlazada mediante clave ajena (FOREIGN KEY ON DELETE CASCADE) al registro principal, preparada para guardar las modificaciones de cada propiedad.

5. Extensiones Implementadas

Para dotar al proyecto de mayor robustez y cumplir con los requisitos avanzados propuestos en la rúbrica, se han implementado las siguientes extensiones:

- A) Extensión 1: Control de Capacidad Computacional (Restricciones OCL)** Se ha incorporado una restricción formal a nivel de metamodelo (R10_NoBuclePropio y límite de carga de trabajo R05) validada mediante OCL. Esto impide diseños ineficientes (como tareas que se apuntan a sí mismas creando bucles) y asegura que un agente no reciba una carga de tareas que exceda los límites operativos.
- B) Extensión 1: Trazabilidad de Datos (Histórico de Contexto SQL y Web)** El sistema cuenta con trazabilidad de datos completa. Se introdujo la clase VersionContexto en el metamodelo. El generador Acceleo traduce esto en una tabla auxiliar _Historial en SQL conectada por clave ajena. Simultáneamente, el generador HTML inyecta un componente visual (Histórico de cambios) en los formularios de la web siempre que una tarea modifique una variable, cumpliendo con los requisitos de auditoría.
- C) Extensión 1: Seguridad y Validación Dinámica Frontend** Se ha enriquecido la clase Propiedad con atributos de configuración de seguridad (extensionPermitida y

pesoMaxMB). En lugar de código estático, Acceleo procesa estos metadatos inyectando de forma automatizada scripts de validación JavaScript en el atributo onchange y restricciones HTML5. Esto protege al sistema de archivos pesados o maliciosos desde la misma fase de modelado gráfico.

D) Extensión 3: Generación Automática de Documentación (Acceleo M2T) Se ha desarrollado un segundo módulo de transformación Acceleo (`GenerateDoc.mtl`) que toma como entrada directa el propio metamodelo (Nivel M2 - `.ecore`). Esta transformación recorre la sintaxis abstracta y extrae dinámicamente las clases, referencias, multiplicidades y, especialmente, las anotaciones de documentación (DSLDoc) y restricciones OCL. El resultado es la generación automática de un portal web estructurado (`McpMM_Documentacion.html`) que actúa como diccionario de datos técnico del lenguaje.

E) Extensión 1: Escalabilidad Jerárquica (Patrón Composite) Para mejorar significativamente la expresividad del lenguaje (Nivel M2), se ha introducido el patrón arquitectónico *Composite* en el elemento raíz `ServiceMCP`, permitiendo que se contenga a sí mismo. A diferencia de una estructura plana básica, esto permite a los ingenieros modelar "Sistemas de Sistemas", anidando sub-servicios y creando jerarquías organizativas complejas.

6. Herramientas de desarrollo

Para el ciclo de vida completo del proyecto (modelado, validación y generación de código), se ha utilizado el ecosistema de **Eclipse Modeling Project**. A continuación se detallan las herramientas y versiones específicas:

Entorno de Desarrollo Integrado (IDE): Eclipse Modeling Tools.

Eclipse Modeling Framework (EMF): Utilizado para la definición de la sintaxis abstracta del DSL mediante el metamodelo Ecore. Versión de cumplimiento (Compliance Level): 6.0.

Emfatic: Lenguaje textual utilizado para agilizar el diseño y la edición del metamodelo `.emf`, permitiendo una definición clara de clases, atributos y relaciones.

EuGENia (Epsilon): Herramienta integrada en el framework Epsilon que automatiza la generación de los modelos necesarios para implementar el editor gráfico GMF (`.gmfgraph`, `.gmftool` y `.gmfmap`) a partir de un único metamodelo Ecore anotado.

Graphical Modeling Framework (GMF): Utilizado para la creación de la sintaxis concreta gráfica, permitiendo al usuario final modelar sistemas mediante diagramas visuales.

Object Constraint Language (OCL):

- **OCLinEcore:** Herramienta para embeber restricciones de validación (invariantes) y lógica de atributos derivados directamente dentro del metamodelo.
- **Interactive OCL Console:** Consola interactiva utilizada para probar y depurar

consultas OCL sobre las instancias de los modelos.

Acceleo: Motor de transformación de modelos a texto (M2T) utilizado para la generación automática de los artefactos finales (HTML para la interfaz web y SQL para la base de datos).

Epsilon (v1.4 o superior): Familia de lenguajes y herramientas para la gestión de modelos que da soporte a EuGENia y otros procesos de validación.

Java Development Kit (JDK): Utilizado para la ejecución del entorno Eclipse y el código generado de los plugins .edit, .editor y .tests.

Github: Para el versionado de código de los integrantes del grupo.

7. Contenidos del archivo que se entrega

Se entregará un archivo comprimido dmss_MisAgentesIA_proyecto.zip que incluye la totalidad del espacio de trabajo (*workspace*) desarrollado y los entregables adicionales exigidos. El contenido exacto es el siguiente:

- **Proyectos Eclipse (Sintaxis Abstracta, Concreta y Generadores):**
 - ProyectoMCP: Proyecto EMF principal que contiene el metamodelo (McpMM.ecore, McpMM.emf), el modelo generador (McpMM.genmodel), así como las carpetas con los modelos de prueba (/modelosPrueba) y el código final generado (/generado y /generadoDoc).
 - ProyectoMCP.edit: Plugin de edición generado automáticamente por EMF.
 - ProyectoMCP.editor: Plugin del editor de árbol reflectivo generado por EMF.
 - ProyectoMCP.tests: Plugin de pruebas generado por EMF.
 - ProyectoMCP.diagram: Proyecto que contiene la definición de la sintaxis concreta gráfica y el editor visual generado mediante el framework EuGENia/GMF.
 - acceleo_mcp: Proyecto de Acceleo que contiene los módulos de transformación Modelo a Texto (M2T). Incluye generate.mtl (encargado de generar el portal web HTML y el script de base de datos SQL) y generateDoc.mtl (encargado de generar la documentación automática a partir del metamodelo).
- **Documentación del proyecto:**
 - dmss_MisAgentesIA_MonteroPajuelo_CarrascoVazquez.pdf: Documento actual con la descripción completa del dominio, las decisiones de diseño, la explicación detallada de las sintaxis abstracta y concreta, y los detalles de la generación de código y extensiones.
- **Vídeo explicativo:**
 - dmss_MisAgentesIA_video.mp4 (o .avi): Demostración práctica en vídeo (duración máxima de 5 minutos) mostrando la creación de un modelo de ejemplo en el editor gráfico, la comprobación de las validaciones OCL, y la ejecución final de las transformaciones Acceleo para visualizar la web, el SQL y la documentación generada.

8. Anexo

Anexo 1. Planificación

El proyecto se inició creando ambos el metamodelo inicial (.ecore) desde el que parte el proyecto, además de generar el diagrama (con el .aird) y el modelo .xmi. Una vez realizado lo anterior decidimos dividirnos el trabajo de transformaciones de modelo, así, uno de la pareja se encargó de la notación gráfica usando GMF y EuGenia (Pablo) y el otro se encargó de generar el código del proyecto usando Acceleo (Javier).

Una vez creado todo lo anterior y probado su funcionamiento, pasamos a refinar detalles del metamodelo, como las restricciones OCL (Pablo) y a adaptar la web generada con Acceleo a las especificaciones que demandaba el proyecto (Javier).

Finalmente, se añadió la extensión número 3 mediante la cual Acceleo genera la documentación de la sintaxis abstracta del metamodelo (Javier) y a la creación de varios modelos de ejemplo.

Anexo 2. Repositorio de Github

Para un mayor acceso a los datos, además de al flujo de trabajo, dejaremos a continuación un enlace al repositorio de Github que hemos usado durante todo el proceso de creación del proyecto. En él se pueden ver las divisiones de trabajo mencionadas anteriormente, así como el contenido modificado, añadido o eliminado en cada sesión de trabajo.

[PabloCarrascoVazquez/ProyectoDMSS_MCP](https://github.com/PabloCarrascoVazquez/ProyectoDMSS_MCP)